

Invoice Audit from A to Z



How five hospital contracts became a checker that reviews 3,942 invoices, flags the wrong ones, says what they should have cost, and admits when it isn't sure.

3,942

invoices checked
for hospitals 2–5

285

invoices flagged as
wrong

58 / 58

wrong invoices
found in the
answer-key hospital

99.2%

of 2,040 planted
errors caught

4.6 h

of the 6–8 hour
time cap

Inside

- The task and the data
- The journey: 5 phases, 12 prompts
- Phase 1 · Understanding the problem
- Phase 2 · Designing the solution
- Phase 3 · Building and verifying the pipeline
- Results for hospitals 2–5
- Phase 4 · Testing it honestly
- Phase 5 · Deliverables and what's next

The task in one page

An insurer pays five hospitals, each under its own contract. Some invoices break their contract. The job: for every invoice from hospitals 2–5, say **whether it's wrong, why, what it should have totalled, and how confident we are**.

5

contracts, each written differently

4,855

invoices across all hospitals

61,211

invoice lines with free-text descriptions

Five hospitals, five contract styles

H1 · Northgate

Clean tables. **Comes with the answer key:** used to build and test, not scored.

H2 · St. Auben

Prose only: ~76 rates hidden in 800 lines of legal filler. Adds a 60-day submission rule.

H3 · Rivermead

Split across **3 documents**. Seven rates change on 1 Jan 2025, by service date.

H4 · Calderwood

Tables with cross-references. Closest to H1.

H5 · Pelham

Prices vary by **facility and plan tier** multipliers, rounded after every step.

What counts as “wrong”

A wrong price or unit, a missing or wrong premium, discount or bundle, a quantity over a daily cap, a duplicate, bad dates or header data.

How it is judged

Quality over coverage

Two hospitals done well beats a thin pass over four.

Honest confidence

A confident wrong answer costs more than an honest “not sure”.

Show your work

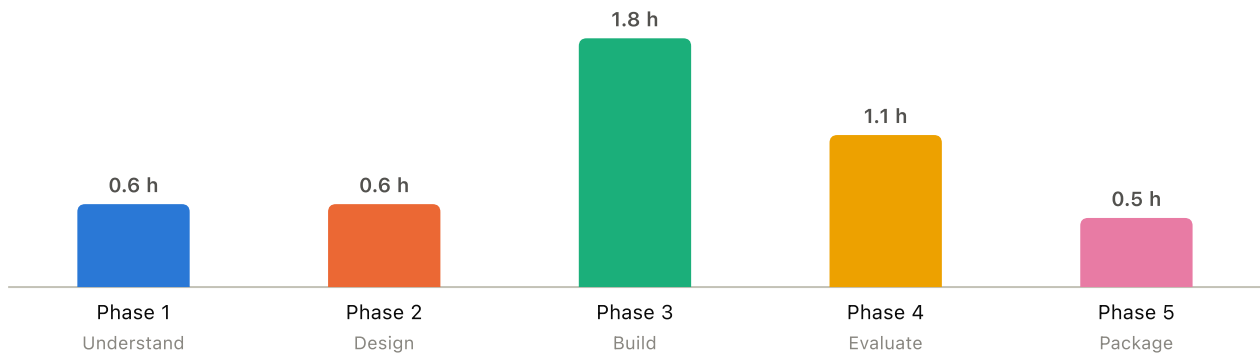
Runnable repo, report, decision log, and every AI prompt kept as a file.

The journey: 5 phases, 12 prompts

Work moved phase by phase. Nothing was built until the plan was agreed, and every prompt was saved word for word.

Time spent per phase

Total 4.6 h of a 6–8 h cap



Every prompt, in order

Dot colour = phase

- | | |
|--------------------------------------|---------|
| ● 001 Understand the task | Phase 1 |
| ● 002 Check my understanding | Phase 1 |
| ● 003 Idea: extract with an LLM | Phase 2 |
| ● 004 Confidence + full plan | Phase 2 |
| ● 005 Is there a free API key? | Phase 2 |
| ● 006 Choose option B | Phase 2 |
| ● 007 Build the pipeline | Phase 3 |
| ● 008 Why high confidence in review? | Phase 3 |
| ● 009 Split confidence, explain | Phase 3 |
| ● 010 Explain phase 4 | Phase 4 |
| ● 011 Evaluate + stress test | Phase 4 |
| ● 012 Final phase | Phase 5 |

The rule we agreed early

If a prediction isn't sure, its confidence must be low, and a real person reviews it.

Moments that changed the plan

LLM-first extraction swapped for code parsers (003) · no free API key, so option B, Claude Code on the subscription (005–006) · one confidence split into flag + amount (008–009).

Phase 1 · What the data really says

Before designing anything, every contract and data file was read and profiled. The answer key for hospital 1 taught us what “wrong” looks like.

58 / 913

hospital 1 invoices are wrong (6.4%)

18

different error categories

13

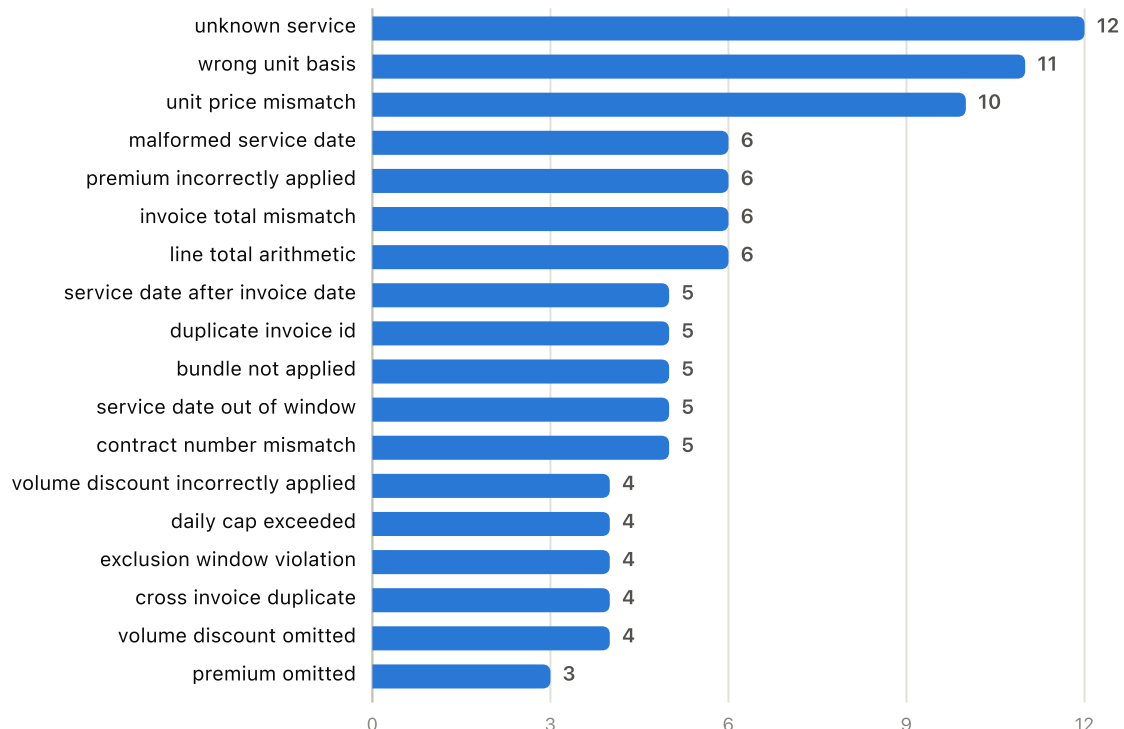
unclear contract clauses found

Key discoveries

- **Use JSONL, not CSV.** When an invoice ID is reused, the CSV mixes two invoices' lines together.
- **Use Markdown, not PDF.** PDF text turns “1.05 1 0.95” into “1.051 0.95”.
- **Not every error is about money.** 17 of 58 wrong invoices have the right total (bad dates, wrong contract number...).
- **Descriptions are messy.** “Svc Preop Ent Steril” = Preoperative Otolaryngologic Sterilisation Service.
- **Unclear clauses,** e.g. a “Service Day” that starts at 07:00 when the data only has dates.

Error categories in the answer key

Hospital 1 labels · number of invoices per category (an invoice can carry several)



Phase 2 · Code where exact, AI where fuzzy

The first idea was to have an LLM read the contracts. A quick test changed the plan: the contracts can be read **exactly** by code. The fuzzy part is the invoice descriptions.

4 of 5

contracts are already clean tables

76

rate clauses found in hospital 2's prose

100%

of those sentences matched by 12 fixed patterns

Decisions agreed

D1 · Read contracts with code

Exact and reproducible. An LLM reads them too, but only as a second reader to cross-check.

D2 · Order of work

Prove it on hospital 1, then 4 → 5 → 3 → 2, from most to least similar.

D3 · Match descriptions

Word-by-word matcher first; the LLM gives a second opinion on the unclear ones.

D4 · Honest confidence

Evidence tiers checked on hospital 1; low confidence goes to a human review queue.

D5 · Unclear clauses

Pick a main reading, re-run with the other reading, lower confidence if the answer changes.

D6 · How to run the LLM

No free API key with a subscription, so Claude Code's scripted mode (claude -p), with answers saved to files.

Who does what

Job	Done by	Why
Read rates and rules from contracts	Code (LLM double-checks)	Must be exact: one wrong digit spreads to every invoice
Match messy descriptions	Code first, LLM second opinion	Fuzzy text is where an LLM helps
Price and check invoices	Code only	60,000+ lines of exact cent arithmetic
Decide when to ask a human	Confidence tiers	Uncertainty must be visible, not hidden

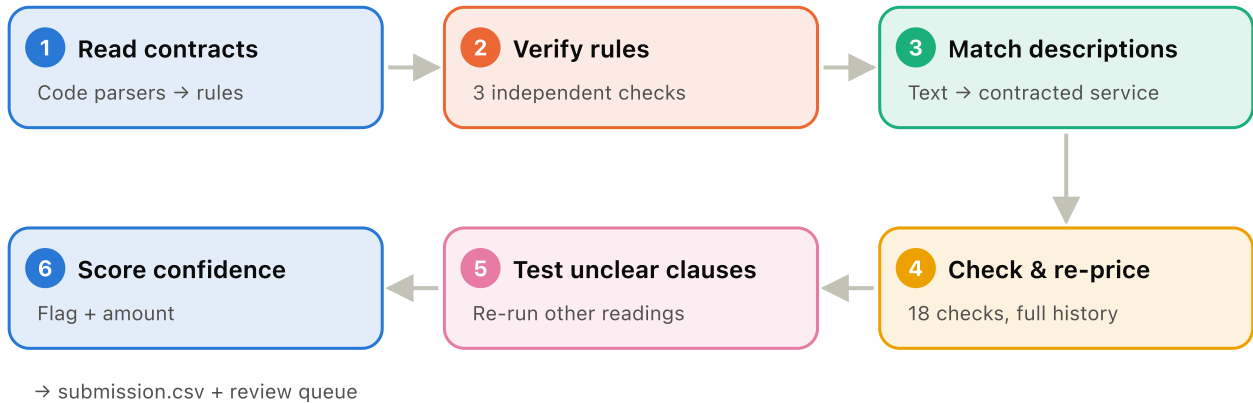
One principle that guards every price check

Never choose a service by its billed price. If the price is the error, you'd pick the wrong service and miss the error.

Phase 3 · How the checker works

One command, `python3 -m audit`, runs everything in about 4 seconds using only Python's standard library. The LLM isn't called during the run: its answers were saved once as files.

Six steps from contract to answer



Were the rules read correctly? Three independent checks

830/830

code self-checks pass (e.g. "eighty" must equal "(80)")

0

differences between the LLM's reading and the code's, all 5 contracts

≥ 96%

of real billed prices equal our calculated price, for every service

Hospital	Services	Self-checks	LLM differences	Lowest price agreement
H1	108	✓ 126/126	✓ 0	97.0%
H2	76	✓ 306/306	✓ 0	97.9%
H3	120	✓ 173/173	✓ 0	96.0%
H4	98	✓ 127/127	✓ 0	96.0%
H5	84	✓ 98/98	✓ 0	97.0%

Planting fake errors (a 1-cent rate change, a wrong multiplier) proved the comparison does catch differences.

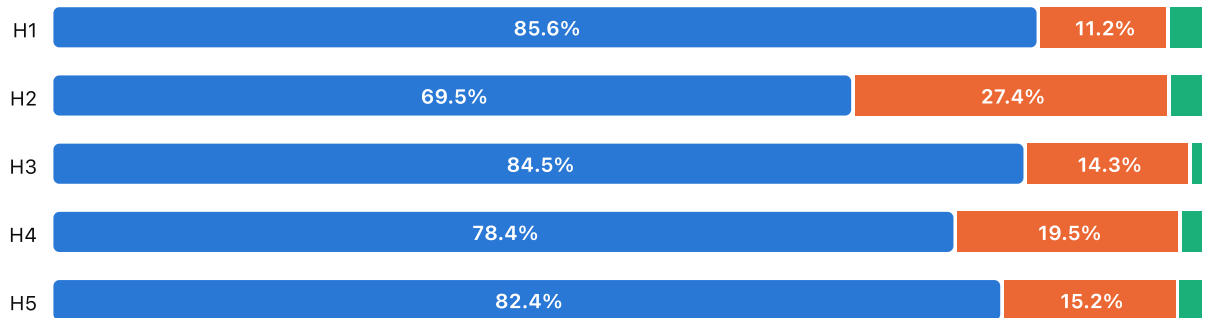
Phase 3 · Reading messy descriptions

Every word in a description must fit a different word of a contracted service ("Spclst" → Specialist, "Ent" → Otolaryngologic). Only one abbreviation out of 194 distinct words needed to be written by hand.

How each invoice line was matched

Share of lines per hospital

Exact match Some words dropped Tie broken by unit or price No match



Lines	Exact	Words dropped	Tie broken	No match
H1	9,770	1,278	355	12
H2	9,982	3,930	434	14
H3	9,843	1,661	137	14
H4	8,277	2,056	214	13
H5	10,899	2,009	301	12

12 / 12

hospital 1's unknown services found by "no match"

556

unclear descriptions given an LLM second opinion (10 LLM calls, ≈\$5 of subscription)

555

agreed with the pipeline; the 1 other was explained by the answer key

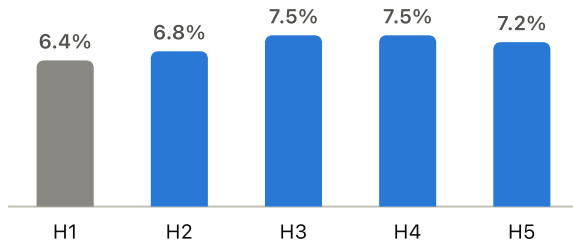
Lessons turned into general rules (not per-invoice fixes)

- A line billed in the wrong unit was still **delivered**, so it still counts for bundles and running totals.
- A loose match whose unit **and** price fit nothing is really an unknown service.
- If several services fit a description, it's a tie, even if one drops fewer words (spotted by the LLM: "Emer Ortho").
- Daily caps: the answer key pays *fewer* units than the cap allows. The contract reading was kept and the amount marked uncertain.

What the checker found in hospitals 2–5

Share of invoices flagged

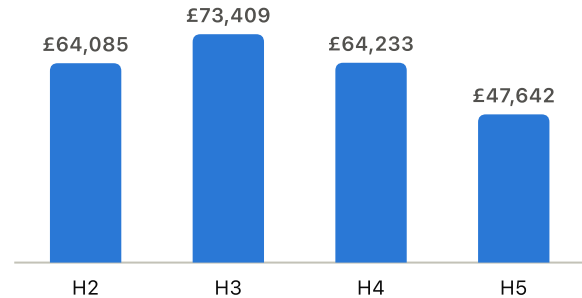
Grey = hospital 1 answer key, for comparison



H1 58/913 · H2 76/1,125 · H3 70/932 · H4 63/835 · H5 76/1,050

Overbilling found in flagged invoices

Billed minus expected total



Error categories flagged

Invoices per category · darker = more

Category	H2	H3	H4	H5
unit price mismatch	13	17	12	23
wrong unit basis	16	14	12	12
unknown service	13	13	9	12
malformed service date	8	7	6	8
invoice total mismatch	8	7	6	7
service date after invoice date	7	7	6	8
daily cap exceeded	8	7	6	7
bundle not applied	7	7	7	6
contract number mismatch	7	6	6	8
line total arithmetic	7	7	6	6
duplicate invoice id	7	7	5	7
service date out of window	6	7	4	8
premium incorrectly applied	7	5	6	7
premium omitted	6	7	6	5
cross invoice duplicate	7	4	4	6
volume discount omitted	6	4	6	5
exclusion window violation	6	3	5	6
volume discount incorrectly applied		5	2	5

Confidence tiers

Invoices per tier (confidence = chance the flag is right)

Tier	Conf.	H2	H3	H4	H5
C1 Clean · no tie-breaks	0.97	706	740	592	719
C2 Clean · unit tie-break	0.95	297	70	149	254
C3 Clean · price tie-break	0.85	46	48	31	0
C6 Clean · depends on a clause reading	0.60	0	4	0	1
F1 Flagged · mechanical finding	0.97	44	40	35	40
F2 Flagged · pricing / identification	0.93	32	30	28	36

Human review queue · 33 invoices

- **28** · amount uncertain: over a daily cap, flag certain
- **5** · flag and amount uncertain: depend on one clause reading

Also caught: hospital 3 billing 2025 amendment rates for 2024 dates, and hospital 5 applying the wrong multiplier.

Phase 4 · A perfect score proves little

Hospital 1 scores 100% in every category, but the checker was built on it. So two extra tests measure how much that score can be trusted.

58 · 0 · 0

hospital 1: found · false alarms · missed

909 / 913

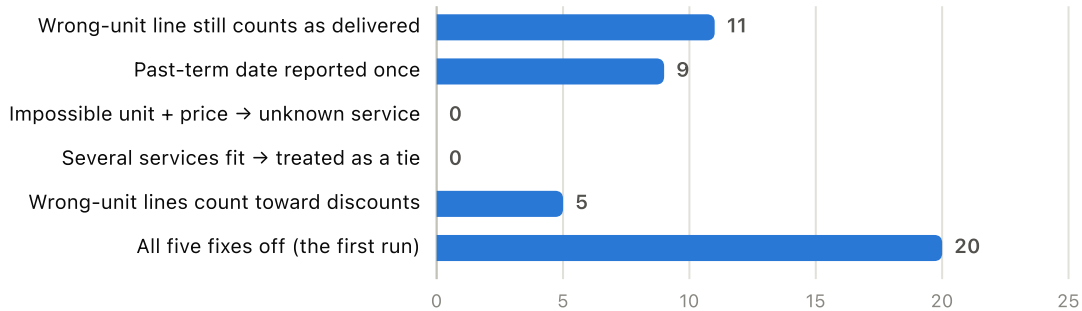
expected totals exact (the 4 misses are cap cases)

20

of 3,942 submitted rows change if every hospital-1 fix is removed

Test 1 · How much did tuning on hospital 1 matter?

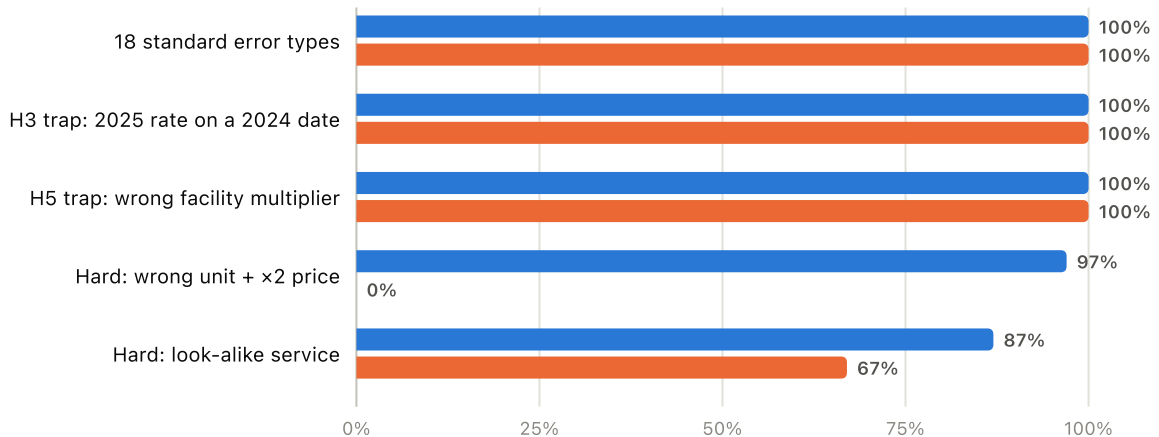
Submitted invoices that would change if one fix were switched off. The very first run already found 58/58 with 0 false alarms.



Test 2 · Stress test: 2,040 known errors planted into all five hospitals

Clean invoices were given one error each, then the full checker ran

■ Flagged ■ Right category



Per hospital, 98.8–100% flagged. All 16 misses landed in the review queue. The two hard types are deliberately tricky.

Phase 4 · Four ways the approach can go wrong

The brief asks for the systematic weaknesses, not a list of misses. Each one was confirmed by replaying a real example.

A Look-alike services

A service **not** in the contract, written with a word dropped, looks like a real one.

Example: "Supervised Dialysis Session" was flagged as a price error instead of an unknown service.

B A wrong unit hides a price

A line billed in the wrong unit is never priced, so a price error on it goes unseen.

Example: Unit changed *and* price doubled: only the wrong unit is reported.

C Knock-on findings

One broken line changes how *other* lines are priced.

Example: A bad date removes a same-day bundle, so a correct price looks wrong (+£294).

D Cap amounts

The contract and the answer key disagree on what to pay above a daily cap.

Example: 11 sessions billed, cap 8: we pay 8, the key pays 3.

A · What happens to look-alike services

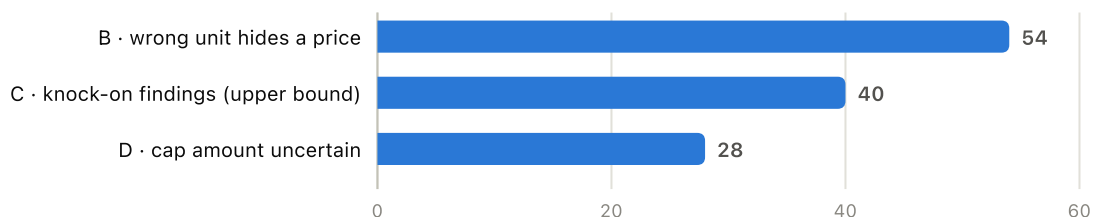
500 planted look-alikes over 5 random seeds · only 1.2% slip through silently

■ Caught as unknown service · 76.2%
 ■ Flagged, other category · 12.8%
 ■ Missed → sent to review · 9.8%
 ■ Missed silently · 1.2%



B–D · Submitted invoices exposed

How many rows each weakness could touch · C is an upper bound: all 10 such cases in hospital 1 were genuine multi-error invoices



Is the confidence honest?

✓ Never overconfident

On hospital 1, every tier was right at least as often as its stated confidence.

✓ Risk lands in the right place

Every missed planted error sat in the lowest clean tier (0.70), which goes to review.

! A known limit

Hospital 1 can't show under-confidence, and false alarms on hospitals 2–5 can't be measured without labels.

Phase 5 • Delivered, and what comes next

All five deliverables

✓ 1 • Runnable repository

One command rebuilds everything; a fresh clone reproduces submission.csv byte for byte; 12 tests pass.

✓ 2 • submission.csv

3,942 rows for hospitals 2–5, in the template format.

✓ 3 • Evaluation report

Per-category results, the stress test, four failure types, calibration.

✓ 4 • Prompts as files

All 12 session prompts word for word, plus the two versioned pipeline prompts.

✓ 5 • Decision log

14 unclear clauses, 7 answer-key conventions, 7 method choices, each with evidence.

Next steps, by impact

- **Ask the payer** which cap-amount convention applies (28 invoices).
- **Price wrong-unit lines** under each possible unit to catch hidden price errors (54 invoices).
- **Version 2 of the mapping prompt**, showing unit and price to catch look-alikes.
- **Mark knock-on findings** as consequences, not separate errors.
- **Spot-check ~100 invoices** from hospitals 2–5 to calibrate on real data.

Where things live

audit/	the pipeline code
submission.csv	the answer file
outputs/	review queue, detailed predictions, evaluation
docs/	understanding, design, evaluation, decisions, next steps
prompts/	every prompt, word for word

Status

Committed on the local submission branch. Not pushed yet: the next step is your own GitHub repository.